Advanced Techniques for EEG Processing - Laboratory Exercises

Exercise 1

The file "sub-035_ses-01_task-Rest_eeg.mat" contains the EEG data relative to a single healthy subject while performing a trial-based experiment. The experiment consists in a *3-stimulus auditory oddball task*. This kind of task is explained in the text of the previous lab activity ('The EEG system and a trial-based experiment'). In the specific 3-stimulus auditory oddball task, the standard stimulus was a 440 Hz sinusoidal tone (70% of trials), the target stimulus was a 660 Hz sinusoidal tone (15% of trials) and the distractor (15% of trials) was a naturalistic sound. Each stimulus lasted 200 ms and a random inter-stimulus interval was used. The subject was instructed to mentally count the target stimuli and ignoring the standard and distractor stimuli. The EEG data were recorded continuously from 59 electrodes using CPz as reference electrode, at a sampling rate of 500 Hz.

The aim of this exercise is to apply some first steps of preprocessing to the EEG data. The .mat file "sub-035_ses-01_task-Rest_eeg.mat" contains the following quantities:
`X`: EEG data organized as a 2-D matrix whit size $n \times m$ where n is the number of electrodes and m the number of time samples.
`ch_names`: cell array containing the labels of the n electrodes.
`srate`: containing the value of the sampling rate.
`stim_samples`: array containing the time samples at which the stimuli were applied
`stim_types`: cell array indicating the type of each applied stimulus (labels: 'standard', 'target', 'distractor')

1) Load the .mat file; transform `X` from single precision (used to reduce the space occupied by the .mat file) to double precision using function `double`.
2) Write instructions to possibly allow resampling of the EEG data. To this aim use the function `resample`. Keep `p` and `q` as parameters assigning them value 1; in this way, no resampling is performed but a user could decide to perform resampling just modifying the values of these parameters (e.g. `p=1` and `q=2` halve the sampling rate). Also, using `p` and `q`, compute the (possibly) new sampling rate and compute the time samples of the stimuli according to the (possibly) new sampling rate (overwrite `srate` and `stim_samples`). Use function `floor` to assure that the recomputed time samples are integer values in case the sampling rate is changed.
3) Linearly detrend the EEG data. Use function `detrend`.
4) Plot the n EEG signals as a function of time for the entire duration of the recording.
5) Compute the power spectral density (PSD) of the n EEG signals using function `pwelch`. Plot the PSD of all electrodes in two figures, using function `subplot` (one figure containing 30 subplots and the other figure 29 subplots). Then, in separate figures plot the PSD of the electrodes F3 (n° 10), F1 (n° 11) and PO3 (n° 53) to visualize them larger.
6) Apply three IIR filters in sequence: 1) a low-pass filter with passband up to 60 Hz, stopband from 80 Hz, passband ripple no more than 0.1 dB and stopband attenuation of at least 40 dB; ii) a high-pass filter with passband from 0.5 Hz, stopband up to 0.01 Hz, and same passband ripple and stopband attenuation as the low-pass filter; 3) a notch filter at 60 Hz and bandwidth = 60/45 Hz = 1.33 Hz.
   For both the low-pass and the high-pass filter, implement an elliptic IIR filter using the functions `ellipord` and `ellip`, and for the notch filter use the function `iirnotch`. For each filter, plot the magnitude and phase response of the filter using `freqz`. For each filter, use function `filtfilt` to filter the signals.
7) Plot the n filtered EEG signals as a function of time for the entire duration of the recording.
8) Compute and plot the PSD of the filtered EEG signals and focus on the PSD of the electrodes F3 (n° 10), F1 (n° 11) and PO3 (n° 53) comparing it with the pre-filtered PSD (point 5 above).

9) Over the time plot of the n filtered EEG signals, plot also the markers indicating the time positions of stimuli presentation, using different colors for the three different types of stimuli.
10) Epoch the continuous EEG signals, extracting epochs with bounds [-0.2, 0.8] (in seconds) around each stimulus presentation, and collect the extracted epochs into a 3-D matrix.
11) Concatenate the epochs in a 2-D matrix; to this aim apply function `reshape` to the previous 3-D matrix.
12) Identify bad channels using the correlation method, computing the correlation of each channel with the nearby channels. To this aim, load the file `nearby_channels.mat`. It stores the cell array `nearby_ch` (59 x 1); the $i-th$ cell of `nearby_ch` contains the array of the indices of the channels surrounding the $i-th$ channel. Identify the bad channels as those behaving as outliers compared to the other channels. Store in the numerical array `index_bad` the index/ces of the bad channel/s. Store in the numerical array `index_good` the indices of the good channels (i.e., all channels not identified as bad).
13) Remove the bad channels.
14) Save in the .mat file `sub-035_PreprocessStep1.mat` the following quantities:
    `X`: 2-D matrix of the data after all the previous preprocessing step, i.e. epoch-concatenated data without bad channels;
    `ch_names`: cell array containing the labels of all the n electrodes (both good and bad electrodes);
    `srate`: value of the sampling rate (possibly a new sample rate in case of resampling);
    `stim_types`: cell array indicating the type of each applied stimulus (labels: 'standard', 'target', 'distractor');
    `index_bad`: the numerical array containing the index/ces of the bad channel/s;
    `index_good`: the numerical array containing the indices of the good channels;

<u>IMPORTANT</u>: A cell array is a data type with indexed data containers called cells, where each cell can contain any type of data. Access the contents of the cells of a cell array by indexing with curly braces `{}`; e.g. to access the content of the i-th cell of the cell array `ch_names`, write `ch_names{i}`; to access the content of the j-th cell of the cell array `nearby_ch`, write `nearby_ch{j}`.

Some useful functions

```
Y=resample(X,p,q)
```

resamples the values `X` of a uniformly sampled signal, at `p/q` times the original sample rate using an antialiasing filter. If `X` is a matrix, then `resample` treats each column as an independent channel.

```
[P,F]=pwelch(X,WINDOW,NOVERLAP,NFFT,Fs)
```

returns in `P` the power spectral density of `X`, estimated by dividing `X` into sections containing a number of samples specified in `WINDOW`. `NOVERLAP` specifies the samples of overlap between adjacent sections (leaving it at the default value [ ], a 50% overlap is used). `NFFT` is the number of points used to calculate the spectrum; if `NFFT` is greater than `WINDOW` the data in each section is zero-padded. `Fs` is the sampling frequency. `F` returns the vector of frequencies at which the power density is estimated (the resolution in Hz is equal to `Fs/NFFT`).

If `X` is a vector, it is converted to a column vector and treated as a single channel. If `X` is a matrix, the power spectral density is calculated independently for each column and saved in the corresponding column of `P`.

`[N, Wp] = ellipord(Wp, Ws, Rp, Rs)`

returns the order `N` of the lowest order digital elliptic filter which has a passband ripple of no more than `Rp` dB and a stopband attenuation of at least `Rs` dB. `Wp` and `Ws` are the passband and stopband edge frequencies, normalized from 0 to 1.

`[b,a] = ellip(N,Rp,Rs,Wp)`

designs an `N`th order lowpass digital elliptic filter with `Rp` dB of peak-to-peak ripple and a minimum stopband attenuation of `Rs` dB. `ellip` returns the filter coefficients in length N+1 vectors `b` (numerator) and `a` (denominator). The passband-edge frequency `Wp` must be 0.0 < `Wp` < 1.0, with 1.0 corresponding to half the sample rate.
To implement a high-pass filter use `[b,a] = ellip(N,Rp,Rs,Wp,'high')`

`freqz(b,a,Npts,Fs)`

with no output arguments plots the magnitude and phase of the filter in the current figure window. The frequency response of the filter is evaluated at `Npts` points equally spaced from 0 to half the sample rate `Fs`. In case `Fs` is not specified, the frequency response is evaluated at `Npts` points equally spaced from 0 to 1. If `Npts` isn't specified, it defaults to 512.

`[b,a] = iirnotch(Wo,BW)`

designs a second-order notch digital filter with the notch at frequency `Wo` and a bandwidth of `BW` at the -3 dB level. `Wo` must satisfy 0.0 < `Wo` < 1.0, with 1.0 corresponding to one half the sample rate.
The bandwidth `BW` is related to the Q-factor of a filter by `BW` = `Wo` /Q.

`Y = filtfilt(b, a, X)`

filters the data in vector `X` with the filter described by vectors `a` and `b` to create the filtered data `Y`.
After filtering in the forward direction, the filtered sequence is then time reversed and run back through the filter; `Y` is the time reverse of the output of the second filtering operation. The result has precisely zero phase distortion. If `X` is a matrix, then `filtfilt` treats each column as an independent channel (thus if your data `X` has channels along the rows, you have to give `X'` as input and then transpose the filtered data i.e. `Y=Y'` so that finally `Y` has the filtered channels along the rows).

Example:

```
%For 1000 Hz data, design a lowpass filter with less than 3 dB of
%ripple in the passband defined from 0 to 40 Hz and at least 60 dB
%of ripple in the stopband defined from 150 Hz to the Nyquist
%frequency (500 Hz):

srate=1000; %Hz
Wp = 40/500; Ws = 150/500;
Rp = 3; Rs = 60;
[N,Wp] = ellipord(Wp,Ws,Rp,Rs)      %Gives mimimum order N of the filter
[b,a] = ellip(N,Rp,Rs,Wp);          %Elliptic filter design
freqz(b,a,512,srate);               %Plots the frequency response
X_lp=filtfilt(b,a,X);               %filter the data X (assuming channels along
                                    %the columns)
```